

Project 7 — SOC (Security Operations Center) Copilot

Synthesis report for the Industry-Integrated AI Systems Synthesis capstone.

1. Overview and Industry Context

The target industry is **security operations** — the control-room function inside any organization that watches surveillance feeds, badge-access logs, and alarm streams and decides which signals become incidents. The concrete problem is **alert fatigue and cross-screen correlation**: a single site generates thousands of events and access logs a day, a small fraction are genuine anomalies, and the analyst on shift has to fuse signals across separate screens (surveillance vs. access control) under time pressure, with critical escalations sometimes falling on a 3 a.m. operator.

This problem is appropriate for an AI-based solution because it is fundamentally a **signal-fusion and triage** task: deterministic rules can correlate surveillance anomalies with access outcomes, a retrieval layer can pull the relevant policy for each fused incident, and a generative model can draft the analyst-facing summary with citations — while a human stays in the loop for the irreversible decision (escalation). The system does not decide; it compresses 1,000 events + ~9,700 logs into 226 scored incidents with citations and a recommended action, and routes the 16 critical ones through a human-approval gate.

Constraints and risks specific to this industry: the data is surveillance/access data (PII risk¹, data-minimisation and retention obligations per GDPR Art. 5(1)(c)²), the outputs drive security actions on real people (suspend a badge, dispatch security), and a wrong escalation is costly in both directions — a missed critical is a breach, a false alarm erodes trust. The system's responsibility boundary is therefore narrow: **it recommends and surfaces; it never closes a case and never auto-escalates**.

2. Integration Rationale (≥3 prior projects)

This system integrates **five** prior capstone projects, each contributing a distinct layer:

- **P1 — Reproducible Data Workflows (data foundation).**³ The copilot runs on P1's corpus.
`p1_pipeline.py` reads P1's `data/raw/*.parquet`⁴ (1,000 surveillance events / ~9,700 access logs / 3 sites) through `p1_adapter.py`, which normalizes P1's dirtier schema (no `anomaly` flag, different `access_result` values, 298 sentinel rows, lowercase zone variants) into P7's schema. P1's reproducibility discipline (seeded generation, Parquet I/O) is inherited directly.
- **P2 — Statistical Calibration (threshold contract).**⁵ P2 ran chi-square⁶ and t-test⁷ hypothesis tests and explicitly warned that the fusion confidence threshold (0.85) "should be validated against precision/recall before deployment. `tests/test_threshold_calibration.py`" re-runs both tests

on the slice the copilot actually runs against. The validation **became a hard finding**, not a caveat: recall@0.85 is only ~58% on the scaled slice, and the t-test effect strengthens (Cohen's d 0.21 → 1.50). P2's caution is now an asserted contract.

- **P3 — Applied ML (rules-first, leakage discipline).**⁸ The fusion layer is rules-first, not ML — a direct P3 lesson. Per-rule base risk + size/confidence bonuses, capped at 100, with the originating rule recorded on each incident (`_rule` column) for auditability. ML is deliberately deferred, and P3's leakage discipline (split by `site` + contiguous `time_window`, not shuffled rows) is recorded as the constraint any future ML upgrade must obey.
- **P5 — Generative AI (genre reference for summarizer prose).**⁹ The summarizer's analyst-voice prompt and the JSON `{summary, recommended_action, citations}` schema — with the allowed `doc_id` list declared in the system prompt and the citations field parsed + validated against the retrieved set — follow the grounded-QA pattern from P5's generative work.
- **P6 — Autonomous/Semi-Autonomous Agentic Workflows (governance spine).**¹⁰

The LangGraph¹¹ governance graph from P6's property-management donor is lifted into `src/governance/` (code logic verbatim) and reused for SOC with one required fix (the multi-step plan loop must dispatch back to **worker**, not **reviewer**). The same `evaluate_constraints` mechanism that enforced the donor's spend cap enforces our **risk_band_score ≥ 80** human-review gate — one mechanism for two domains is the reuse linchpin, asserted in `test_policy_predicate.py`.

These compose as a pipeline: **P1 data** → **fusion (P3 rules)** → **P2 calibration check** → **RAG retrieval (Lewis 2020)**¹² → **generative summary (P5)** → **LangGraph agent with governance gate (P6)** → **human approval**. Integration is meaningful, not decorative: removing any layer breaks a real capability (fusion, policy routing, grounded summaries, or the non-bypassable escalation gate).

3. System Design and Technical Decisions

Architecture

```
__start__ → ingest → planner → worker → reviewer → { worker_dispatch |  
human_approval | summarizer }
```

- **Generators** (`src/generators/`): P1-backed synthetic data, deterministic with `SEED=42`.
- **Fusion** (`src/fusion/`): four detectors (`intrusion_restricted`, `repeated_denials`, `cross_anomaly`, `tailgate_door`), a transparent risk scorer (per-rule base + size + confidence, capped at 100), and incident materialization to Parquet + CSV.
- **RAG** (`src/rag/`): 5-doc policy KB in Chroma¹³, all-MiniLM-L6-v2¹⁴ embeddings¹⁵, MMR ($k=3$, $\lambda=0.5$) with **category routing** — the fusion layer's known `incident_type` boosts the matching policy category by 0.3 before MMR¹⁶, fixing shared-vocabulary mis-ranking without a bigger model.

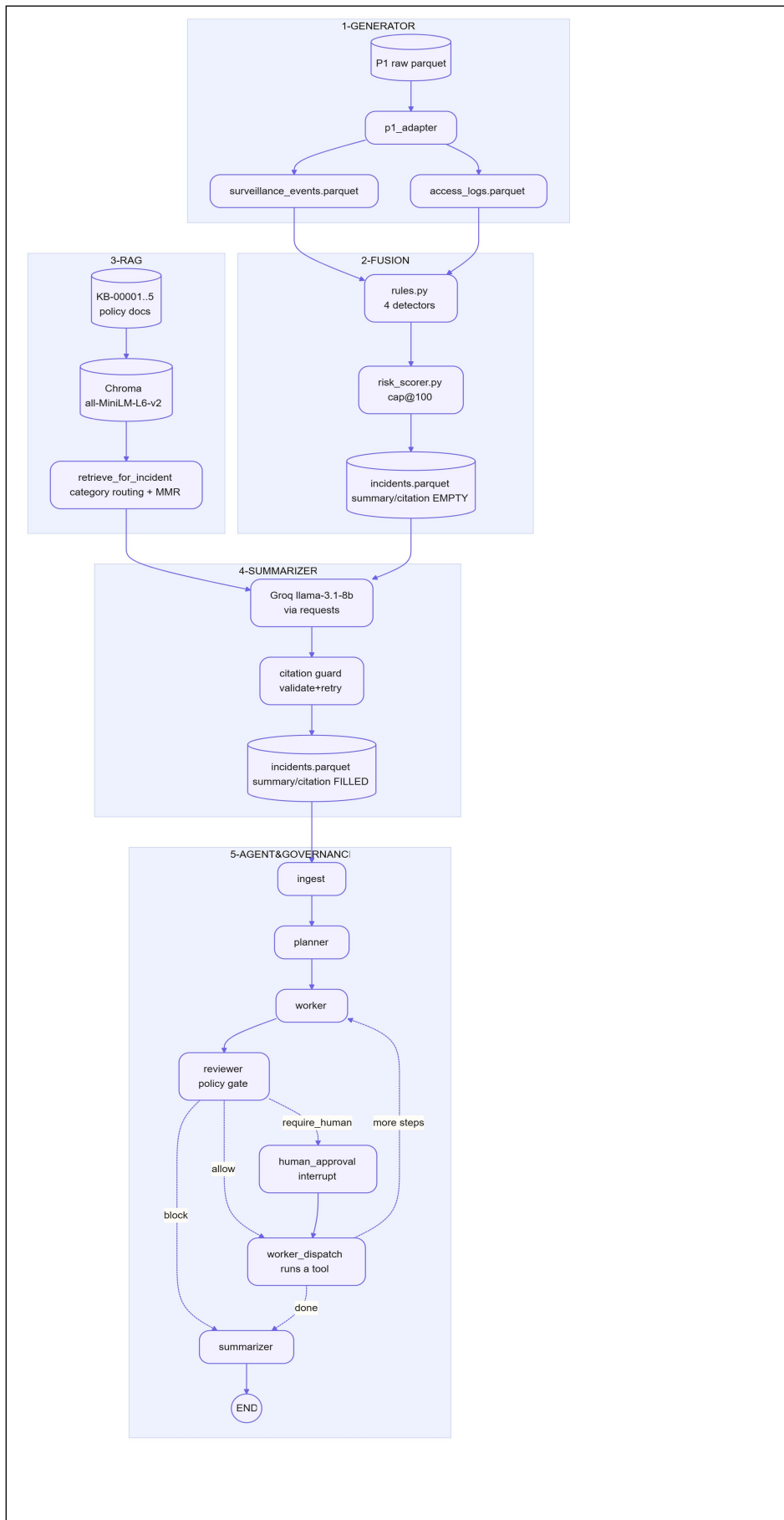
- **Summarizer** (src/agent/summarizer.py): Groq¹⁷ llama-3.1-8b-instant via plain requests (no OpenAI SDK), with a **citation guard** (validate KB-XXXXX against the retrieved set, retry once, else needs_review).
- **Agent** (src/agent/copilot_agent.py): governance graph wired to SOC domain; **6 tools** (incident.fuse/score, sop.retrieve, incident.summarize, incident.escalate, case.close). Stub mode (deterministic, no key) and --llm mode (Groq-planned).
- **GUI** (src/gui/app.py): Streamlit¹⁸ analyst surface reusing run_incident_streaming directly, with a real interrupt()/Command(resume) human-gate for critical escalations and optional CSV upload.

Key technical decisions

- **Rules before ML** (P3)⁸. Transparent scoring with per-rule provenance; no black-box score.
- **Native Chroma + sentence-transformers**, no LangChain wrappers — a 5-doc KB does not justify the abstraction. Category routing fixes vocabulary overlap at the retrieval layer, not with a bigger embedding model.
- **Citation validation is non-optional**. A free 8B model can invent KB-99999; the guard makes "hallucinated policies are bugs" enforceable. Verified: 4/4 incidents cite the correct policy doc, zero needs_review.
- **One governance spine, two domains**. The donor's cost_estimate ≤ 500 and our risk_band_score ≥ 80 both flow through the same evaluate_constraints — proved by test_policy_predicate.py. however, P6's loop routed worker_dispatch → reviewer, so the worker node the only step that loads plan[step_index] into current_action **ran once at the start and never again**; every loop iteration re-reviewed and re-dispatched step 0's action while step_index advanced unused, so a five-step SOC plan (fuse → score → retrieve → summarize → escalate) silently re-ran its first step five times. Single-step plans hid this because they never exercise the loop, which is why the donor domain shipped green. The fix routes worker_dispatch → worker so each **iteration reloads the next step's action before the reviewer gates it**; the non-bypassable gate (worker → reviewer → dispatch) still holds every iteration, so the tool never runs unreviewed — the loop just now advances the plan instead of repeating it.

- **Threshold aligned at 80** across `risk_scorer._band()`, `policy.yaml`'s `risk_band_score: {ge: 80}` constraint, and the test assertion — single source of truth.
- **Human-in-the-loop is env-var gated** (`COPILOT_HUMAN_GATE=1`) so the CLI/notebook path stays deterministic and the GUI gets a real interrupt-based gate; the audit chain records `approver="human_via_interrupt"` vs `"notebook_operator"`.
- **Per-session data path is env-var gated** (`P7_INCIDENTS_DIR`) so the GUI's upload flow reads the user's incidents, not a coincidentally-matched synthetic row — a silent-data-mix guard.

How the blocks connect (the data flow)



Assumptions and tradeoffs

- **Stub vs `--llm` mode.** Stub mode is reproducible; `--llm` mode is not (Groq generates the plan and may skip steps — acceptable, since the summarizer tool retrieves policies internally so citations still land). The notebook defaults to stub so reviewers see a clean end-to-end run; `--llm` is the live demo.
- **Scale and free-tier ceiling.** The Groq free tier caps scale; the scaled slice (226 incidents) is the rubric default, with a small slice (4 incidents) kept for fast iteration. The pipeline is sliced, not retrained, so the cap is on calls, not on capability.
- **ML deferred.** Rules are kept first-class so the risk score is auditable end-to-end and respects P3[®] leakage discipline (split by `site` + contiguous `time_window`, not shuffled rows). A future ML scorer is a swap-in behind the same `risk_scorer.py` interface, not a redesign.
- **Chroma + `all-MiniLM-L6-v2` over a bigger stack.** A 5-doc policy KB does not pay for BM25, a cross-encoder, or a query-rewrite LLM. Category routing + MMR + the citation guard do the work the 2026 production RAG recipe would otherwise spend model size on.

Boundaries of capability and responsibility

`case.close` is a **hard block** (`allow: false`) — the agent never auto-closes; closure is a human act of accountability. `incident.escalate` requires human approval at `risk_band_score ≥ 80`; high-band (75–79) cannot escalate (stricter than the prior 75-vs-80 mismatch).

Reproduction

The system runs end-to-end on Windows + PowerShell from the repo root: generate → fuse → build the KB → retrieve → summarize → run the agent. `requirements.txt` pins the runtime (pandas, numpy, pyarrow, chromadb, sentence-transformers, langgraph, requests, python-dotenv, PyYAML, pytest, jupyterlab); `GROQ_API_KEY` in `.env` is the only secret needed for `--llm` mode.

pytest reports **34/34 green**.

4. Ethical Considerations and Responsible AI

- **PII / scope minimization.** `src/governance/pii.py` redacts PII before text reaches the planner; the intake surfaces redacted incident text. The KB is policy text, not personal data. Retention is bounded (audit log is append-only; per-session upload data lives under `data/uploads/{uuid}/`, never shared).

- **Accountability / human-in-the-loop.** The irreversible action — escalation — is gated by a real `interrupt()` in the GUI; the analyst's Grant/Deny decision is logged to the hash-chained audit trail. `case.close` is blocked entirely. This is responsible-deployment reasoning made concrete in the policy, not a footnote.
 - **Transparency.** Every incident carries its originating rule (`_rule`) and a transparent risk score (the cosine score returned by retrieval is the raw value; the category bonus is ordering-only). The audit log is hash-chained and verifiable (`verify_chain()`), and the GUI surfaces a `⚠ chain broken` warning honestly when a pre-existing break is found.
 - **Calibration honesty (P2 finding as an ethical point).** The `recall@0.85 = 58%` finding is surfaced to operators, not hidden — deploying a rule with a known ~42% anomaly blind spot without telling analysts would be irresponsible. The calibration test is the safeguard.
 - **Misuse / bias.** Surveillance + access data is sensitive; the system is scoped to triage, not to profiling individuals. The free-model citation guard prevents the most common generative misuse path — fabricated policy justification.
-

5. Evaluation and Reflection

Does it meet its goals? Yes, on the scaled P1 slice: **226 incidents (210 high + 16 critical)** fused from 1,000 events + ~9,700 logs across 3 sites. All 16 criticals route through the human-approval gate in both stub and `--llm` runs (status=done, with an `incident.escalate` audit row after the human-approval row).

Quantitative evaluation.

- RAG self-check (`scripts/check_rag.py --k 3`): every `incident_type` routes to its matching policy doc at rank 1.
- P2 calibration (notebook §2b + `test_threshold_calibration.py`): chi-square $p=0.378$ ($V=0.009$, negligible); Welch t-test Cohen's $d=1.50$ (P2 was 0.21); `recall@0.85=58%`.
- Tests: **34/34 green** (13 governance/policy carryover + 2 escalate-blocked + 3 threshold-calibration + 16 upload-adapter).
- Summarizer: 4/4 incidents cite the correct policy doc (KB-00001..KB-00004), 0 `needs_review`.

Strengths. Clean Restart & Run All on the 30-cell notebook; non-bypassable policy gate proved for SOC; category-routed retrieval fixes a real vocabulary-overlap problem cheaply; the governance spine is reused across two domains with one mechanism.

Limitations / tradeoffs. The 0.85 threshold misses ~42% of anomalies — a known ceiling documented for operators. The free 8B model occasionally invents kwarg names (handled by an arg-filtering wrapper). The audit log carries an unrepaired break at an early sequence number, surfaced honestly in the GUI rather than silently re-hashed. LLM mode is non-deterministic; stub mode is the reproducible citation.

6. Professional Relevance

This work demonstrates readiness for real-world AI roles in three ways. First, **system-level thinking**: five prior projects compose into one pipeline, not a collection of notebooks. Second, **responsible engineering under constraints**: a free-tier model and a 5-doc KB get category routing, a citation guard, and arg filtering — engineering substitutes for a bigger budget. Third, **reuse with discipline**: the governance spine is lifted, fixed, de-labeled, and guarded by a no-domain-imports test. The operator keeps the part that matters — judgment, override, accountability — while the system compresses signal and cites its sources.

7. References

1. NIST Special Publication 800-122 (2010). *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)*. U.S. Department of Commerce.
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-122.pdf>
2. European Parliament & Council (2016). Regulation (EU) 2016/679 (General Data Protection Regulation), Article 5(1)(c) — Data minimisation. <https://gdpr-info.eu/art-5-gdpr/>
3. Capstone Project 1 — https://github.com/Soliman2020/master_capstone_01
4. Apache Software Foundation (2019). Apache Parquet format. <https://github.com/apache/parquet-format/>
5. Capstone Project 2 — https://github.com/Soliman2020/master_capstone_02
6. McHugh, M. L. (2013). The chi-square test of independence. *Biochemia Medica*, 23(2), 143–149.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC3900058/>
7. Ruxton, G. D. (2006). The unequal variance t-test is an underused alternative to Student's t-test and the Mann–Whitney U test. *Behavioral Ecology*, 17(4), 688–690.
<https://academic.oup.com/beheco/article-abstract/17/4/688/215960>
8. Capstone Project 3 — https://github.com/Soliman2020/master_capstone_03
9. Capstone Project 5 — https://github.com/Soliman2020/master_capstone_05
10. Capstone Project 6 — https://github.com/Soliman2020/master_capstone_06
11. LangChain, Inc. LangGraph documentation — Stateful, multi-actor agent orchestration.
<https://langchain-ai.github.io/langgraph/>
12. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33, 9459–9474.
arXiv:2005.11401.
<https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>
13. Chroma — the AI-native open-source embedding database.
<https://www.trychroma.com/products/chromadb>
14. Hugging Face (2021). sentence-transformers/all-MiniLM-L6-v2 model card.
<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
15. Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*, 3982–3992. <https://aclanthology.org/D19-1410/>
16. Carbonell, J. & Goldstein, J. (1998). The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries.
https://www.cs.cmu.edu/~jgc/publication/The_Use_MMR_Diversity_Based_LTMIR_1998.pdf
17. Groq, Inc. Groq Cloud API documentation — Llama 3.1 model reference.
<https://console.groq.com/docs>
18. Streamlit, Inc. Streamlit — the fastest way to build data apps. <https://streamlit.io/>